

Ασκήσεις προετοιμασίας (2025-2026)

Κεφάλαιο 12 (Αντικειμενοστραφής Προγραμματισμός)

Ερώτημα 1

Κάντε την αντιστοίχιση των στοιχείων της αριστερής στήλης στα στοιχεία της δεξιάς στήλης:

1. Μήνυμα	A. Υποκλάση
2. Πλήρης συλλογή μεθόδων ενός αντικειμένου	B. Αφηρημένη κλάση (abstract class)
3. Στιγμιότυπο κλάσης	Γ. Υπερκλάση
4. Μια κλάση που κληρονομεί από μια άλλη κλάση	Δ. Διεπαφή αντικειμένου
5. Μια κλάση από την οποία κληρονομεί μια άλλη κλάση	E. Παράκαμψη (override)
6. Τροποποίηση μεθόδου που κληρονομείται από μια κλάση από μια υποκλάση της	ΣΤ. Αντικείμενο
7. Κλάση για την οποία δεν μπορούν να δημιουργηθούν αντικείμενα	Z. Κλήση μεθόδου

Απάντηση:

1. Μήνυμα	Z. Κλήση μεθόδου
2. Πλήρης συλλογή μεθόδων ενός αντικειμένου	Δ. Διεπαφή αντικειμένου
3. Στιγμιότυπο κλάσης	ΣΤ. Αντικείμενο
4. Μια κλάση που κληρονομεί από μια άλλη κλάση	A. Υποκλάση
5. Μια κλάση από την οποία κληρονομεί μια άλλη κλάση	Γ. Υπερκλάση
6. Τροποποίηση μεθόδου που κληρονομείται από μια κλάση από μια υποκλάση της	E. Παράκαμψη (override)
7. Κλάση για την οποία δεν μπορούν να δημιουργηθούν αντικείμενα	B. Αφηρημένη κλάση (abstract class)

Ερώτημα 2

Να υλοποιηθεί στην Python μια κλάση Book με ιδιότητες title, author και year_published. Η κλάση να περιλαμβάνει μέθοδο __str__() που επιστρέφει μια περιγραφή του βιβλίου και μια μέθοδο που ελέγχει αν το βιβλίο είναι νεότερο από ένα δοθέν έτος. Στη συνέχεια, να δημιουργηθούν 2 αντικείμενα της κλάσης και να εμφανιστούν τα χαρακτηριστικά τους.

Απάντηση:

<pre>class Book: def __init__(self, title, author, year_published): self.title = title self.author = author self.year_published = year_published def __str__(self): return f"{self.title}, {self.author}, {self.year_published}" def is_newer_than(self, year): return self.year_published > year book1 = Book("Python 101", "John Doe", 2020) book2 = Book("Algorithms", "Maria Papadopoulou", 2015) print(book1) print("Νεότερο από το 2018;", book1.is_newer_than(2018))</pre>

```
print(book2)
print("Νεότερο από το 2018;", book2.is_newer_than(2018))
```

Ερώτημα 3

Στην Python, να γραφεί η κλάση Book με ιδιότητες title και price. Η κλάση να περιλαμβάνει τη μέθοδο `__repr__()` η οποία θα επιστρέφει περιγραφή της μορφής: `Book(title=<τίτλος>, price=<τιμή>)`. Στη συνέχεια, να γραφεί η κλάση Library, η οποία θα περιέχει μία λίστα από αντικείμενα Book και μια μέθοδο με όνομα `total_value()`, η οποία θα υπολογίζει και θα επιστρέφει το συνολικό κόστος όλων των βιβλίων. Να δημιουργηθούν 3 αντικείμενα Book, να προστεθούν σε ένα αντικείμενο Library και να εμφανιστεί το συνολικό κόστος.

Απάντηση:

```
class Book:
    def __init__(self, title, price):
        self.title = title
        self.price = price

    def __repr__(self):
        return f"Book(title={self.title}, price={self.price})"

class Library:
    def __init__(self):
        self.books = []

    def total_value(self):
        return sum(book.price for book in self.books)

book1 = Book("Python", 20)
book2 = Book("Algorithms", 35)
book3 = Book("Databases", 25)

library = Library()
library.books.append(book1)
library.books.append(book2)
library.books.append(book3)

print(library.books)
print("Συνολικό κόστος:", library.total_value())
```

Ερώτημα 4

Να υλοποιηθεί η κλάση Engine, η οποία θα περιλαμβάνει ιδιότητες όπως horsepower, type (π.χ. "petrol", "diesel", "electric") και manufacturer. Επιπλέον, να περιλαμβάνει μέθοδο `get_specs()` που επιστρέφει περιγραφή των χαρακτηριστικών του κινητήρα. Στη συνέχεια, να υλοποιηθεί η κλάση Car, η οποία θα έχει ιδιότητες όπως make, model, και year, καθώς και μια ιδιότητα engine που θα είναι αντικείμενο της κλάσης Engine. Η κλάση Car να περιλαμβάνει μέθοδο `get_description()` που να επιστρέφει πληροφορίες για το αυτοκίνητο και να ενσωματώνει τα δεδομένα του κινητήρα μέσω της μεθόδου `get_specs()`. Να δημιουργηθεί ένα αντικείμενο Car και να κληθεί η μέθοδος `get_description()`.

Απάντηση:

```
class Engine:
    def __init__(self, horsepower, type, manufacturer):
        self.horsepower = horsepower
```

```

self.type = type
self.manufacturer = manufacturer

def get_specs(self):
    return f"{self.horsepower} hp, {self.type}, {self.manufacturer}"

class Car:
    def __init__(self, make, model, year, engine):
        self.make = make
        self.model = model
        self.year = year
        self.engine = engine

    def get_description(self):
        return f"{self.make} {self.model} ({self.year}) - Engine: {self.engine.get_specs()}"

engine = Engine(150, "petrol", "Toyota")
car = Car("Toyota", "Corolla", 2022, engine)

print(car.get_description())

```

Ερώτημα 5

Να υλοποιηθεί η κλάση `Movie` με ιδιότητες `title`, `director` και `release_year`. Η κλάση να περιλαμβάνει τη μέθοδο `__str__()` η οποία θα επιστρέφει μια περιγραφή της ταινίας ως εξής:

«Η ταινία <τίτλος> σε σκηνοθεσία <σκηνοθέτης> κυκλοφόρησε το <έτος>»

Στη συνέχεια, να υλοποιηθεί η κλάση `MovieCollection`, η οποία θα έχει μία λίστα από ταινίες, καθώς και μέθοδο `movies_before(year)`, η οποία θα επιστρέφει όλες τις ταινίες της συλλογής που κυκλοφόρησαν πριν από το δοθέν έτος. Να δημιουργηθούν 5 αντικείμενα της κλάσης `Movie`, να προστεθούν σε ένα αντικείμενο `MovieCollection`, και να εμφανιστούν οι πληροφορίες των ταινιών που κυκλοφόρησαν πριν από το 2000.

Απάντηση:

```

class Movie:
    def __init__(self, title, director, release_year):
        self.title = title
        self.director = director
        self.release_year = release_year

    def __str__(self):
        return f"Η ταινία {self.title} σε σκηνοθεσία {self.director} κυκλοφόρησε το {self.release_year}"

class MovieCollection:
    def __init__(self):
        self.movies = []

    def movies_before(self, year):
        return [movie for movie in self.movies if movie.release_year < year]

collection = MovieCollection()

collection.movies.append(Movie("The Matrix", "Wachowskis", 1999))

```

```
collection.movies.append(Movie("Titanic", "James Cameron", 1997))
collection.movies.append(Movie("Inception", "Christopher Nolan", 2010))
collection.movies.append(Movie("Pulp Fiction", "Quentin Tarantino", 1994))
collection.movies.append(Movie("Interstellar", "Christopher Nolan", 2014))

for movie in collection.movies_before(2000):
    print(movie)
```

Ερώτημα 6

Να υλοποιηθεί η κλάση Person με ιδιότητες name και age. Η κλάση να περιλαμβάνει τη μέθοδο __str__() που να επιστρέφει περιγραφή του ατόμου ως εξής: «Όνομα: <όνομα>, Ηλικία: <ηλικία> ετών.» . Στη συνέχεια, να υλοποιηθεί η κλάση Student, η οποία να κληρονομεί από την κλάση Person και να περιλαμβάνει επιπλέον ιδιότητα grades (λίστα ακέραιων αριθμών). Η κλάση Student να περιλαμβάνει μέθοδο average_grade() που να υπολογίζει και να επιστρέφει τον μέσο όρο των βαθμών του φοιτητή. Να δημιουργηθούν 3 αντικείμενα τύπου Student και να εμφανιστεί η περιγραφή τους και ο μέσος όρος βαθμών τους.

Απάντηση:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"Όνομα: {self.name}, Ηλικία: {self.age} ετών."

class Student(Person):
    def __init__(self, name, age, grades):
        super().__init__(name, age)
        self.grades = grades

    def average_grade(self):
        return sum(self.grades) / len(self.grades)

students = [
    Student("Νίκος", 20, [8, 7, 9]),
    Student("Μαρία", 21, [10, 9, 8]),
    Student("Ελένη", 19, [6, 7, 8])
]

for student in students:
    print(student)
    print("Μέσος όρος:", student.average_grade())
```

Ερώτημα 7

Να υλοποιηθεί η κλάση Product με ιδιότητες name και price. Η κλάση να περιλαμβάνει τη μέθοδο __repr__() η οποία θα επιστρέφει περιγραφή της μορφής: «Product(name=<όνομα>, price=<τιμή>)». Στη συνέχεια, να υλοποιηθεί η κλάση ShoppingCart, η οποία θα έχει ιδιότητα μία λίστα από αντικείμενα Product και μέθοδο total_price(), η οποία θα υπολογίζει και θα επιστρέφει το συνολικό κόστος όλων των προϊόντων. Να δημιουργηθούν 4 αντικείμενα Product, να προστεθούν σε ένα ShoppingCart και να εμφανιστεί το συνολικό κόστος.

Απάντηση:

```

class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def __repr__(self):
        return f"Product(name={self.name}, price={self.price})"

class ShoppingCart:
    def __init__(self):
        self.products = []

    def total_price(self):
        return sum(product.price for product in self.products)

cart = ShoppingCart()
cart.products.append(Product("Laptop", 800))
cart.products.append(Product("Mouse", 20))
cart.products.append(Product("Keyboard", 50))
cart.products.append(Product("Monitor", 150))
print("Συνολικό κόστος:", cart.total_price())

```

Ερώτημα 8

Να υλοποιηθεί η κλάση `Course` με ιδιότητες `title` και `hours`. Η κλάση να περιλαμβάνει μέθοδο `__init__()` για την αρχικοποίηση των ιδιοτήτων και μέθοδο `__repr__()` η οποία να επιστρέφει περιγραφή της μορφής: «`Course(title='<τίτλος>', hours=<ώρες>)`». Στη συνέχεια, να υλοποιηθεί η κλάση `Curriculum`, η οποία να περιέχει μία λίστα από αντικείμενα `Course` και μέθοδο `total_hours()`, η οποία να επιστρέφει το συνολικό αριθμό ωρών διδασκαλίας όλων των μαθημάτων. Να δημιουργηθούν 3 αντικείμενα `Course`, να προστεθούν σε ένα `Curriculum` και να εμφανιστούν οι περιγραφές τους και το σύνολο των ωρών διδασκαλίας.

Απάντηση:

```

class Course:
    def __init__(self, title, hours):
        self.title = title
        self.hours = hours

    def __repr__(self):
        return f"Course(title='{self.title}', hours={self.hours})"

class Curriculum:
    def __init__(self):
        self.courses = []

    def total_hours(self):
        return sum(course.hours for course in self.courses)

curriculum = Curriculum()
curriculum.courses.append(Course("Python", 30))
curriculum.courses.append(Course("Databases", 40))
curriculum.courses.append(Course("Algorithms", 45))
print(curriculum.courses)
print("Σύνολο ωρών:", curriculum.total_hours())

```

Ερώτημα 9

Στην Python ορίστε την κλάση `Weight` έτσι ώστε να δημιουργούνται αντικείμενα βαρών σε κιλά ή σε λίβρες με εντολές της μορφής:

```
Weight(3.5, "KG")
```

```
Weight(3.5, "LB")
```

Ορίστε τη μέθοδο `__str__` της `Weight`. Υπερφορτώστε τον τελεστή `+` έτσι ώστε να αθροίζονται δύο αντικείμενα `Weight` και να προκύπτει ένα νέο αντικείμενο. Το αποτέλεσμα να είναι στις μονάδες του αριστερού τελεστέου της πράξης. Δίνεται ότι 1 κιλό ισούται με 2.2046 λίβρες. Συμπληρώστε τον ακόλουθο κώδικα στη θέση του `pass` έτσι ώστε να εμφανίζει αυτά που φαίνονται δεξιά:

<pre>class Weight: pass if __name__ == "__main__": w1 = Weight(10, "KG") print(w1) w2 = Weight(5, "LB") print(w2) print(w1 + w1) print(w1 + w2) print(w2 + w1)</pre>	<pre>10.0000 KG 5.0000 LB 20.0000 KG 12.2680 KG 27.0460 LB</pre>
Κώδικας (να συμπληρωθεί)	Επιθυμητή έξοδος

Απάντηση:

<pre>class Weight: def __init__(self, value, unit): self.value = value self.unit = unit def __str__(self): return f"{self.value:.4f} {self.unit}" def __add__(self, other): if self.unit == other.unit: return Weight(self.value + other.value, self.unit) if self.unit == "KG": return Weight(self.value + other.value / 2.2046, "KG") else: return Weight(self.value + other.value * 2.2046, "LB") if __name__ == "__main__": w1 = Weight(10, "KG") print(w1) w2 = Weight(5, "LB") print(w2) print(w1 + w1) print(w1 + w2) print(w2 + w1)</pre>

Ερώτημα 10

Δίνεται ο ακόλουθος κώδικας C++:

<pre>#include <iostream></pre>

```
using namespace std;
class Base {
public:
    virtual void show() { cout << "Arta" << endl; }
};
class Derived : public Base {
public:
    void show() override { cout << "Ioannina" << endl; }
};
int main() {
    Derived obj;
    Base *ptr = &obj;
    ptr->show();
}
```

Τι θα εμφανίσει το πρόγραμμα όταν εκτελείται; Ποιος είναι ο ρόλος της λέξης κλειδί virtual στον κώδικα;
Τι θα συμβεί αν αφαιρεθεί η λέξη virtual από τη δήλωση της συνάρτησης show στη κλάση Base;

Απάντηση:

Θα εμφανίσει:
Ioannina

Η λέξη virtual επιτρέπει τον πολυμορφισμό. Όταν μια μέθοδος είναι virtual, η C++ αποφασίζει ποια έκδοση της μεθόδου θα καλέσει κατά την εκτέλεση και όχι κατά τη μεταγλώττιση.

Αν αφαιρεθεί η λέξη virtual τότε θα εμφανίσει:
Arta

Ερώτημα 11

Δίνεται ο παρακάτω κώδικας C++:

```
#include <iostream>
#include <string>
using namespace std;

class BankAccount {
private:
    string owner;
    double balance;

public:
    BankAccount(string owner, double balance) {
        this->owner = owner;
        this->balance = balance;
    }

    void deposit(double amount) {
        balance += amount;
    }

    bool withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
            return true;
        }
        return false;
    }

    void show() {
```

```

        cout << "Owner: " << owner << endl;
        cout << "Balance: " << balance << endl;
    }
};

int main() {
    BankAccount account("Maria", 1000);
    account.deposit(250);
    account.withdraw(400);
    account.show();
    return 0;
}

```

Να μετατραπεί σε ισοδύναμο πρόγραμμα Python.

Απάντηση:

```

class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            return True
        return False

    def show(self):
        print("Owner:", self.owner)
        print("Balance:", self.balance)

account = BankAccount("Maria", 1000)
account.deposit(250)
account.withdraw(400)
account.show()

```

Ερώτημα 12

Δίνεται ο παρακάτω κώδικας Python:

```

class Book:
    def __init__(self, title, author, price):
        self.title = title
        self.author = author
        self.price = price

    def is_expensive(self):
        return self.price > 50

book = Book("Python Basics", "John Smith", 45)
print(book.is_expensive())

```

Να μετατραπεί σε ισοδύναμο πρόγραμμα C++.

Απάντηση:

```
#include <iostream>
```



```

#include <string>
using namespace std;

class Book {
private:
    string title;
    string author;
    double price;

public:
    Book(string title, string author, double price) {
        this->title = title;
        this->author = author;
        this->price = price;
    }

    bool is_expensive() {
        return price > 50;
    }
};

int main() {
    Book book("Python Basics", "John Smith", 45);
    cout << book.is_expensive() << endl;
    return 0;
}

```

Ερώτημα 13

Δίνεται ο παρακάτω κώδικας Python:

```

class Student:
    def __init__(self, name, grades):
        self.name = name
        self.grades = grades

    def average(self):
        return sum(self.grades) / len(self.grades)

student = Student("Maria", [8, 9, 10])
print(student.average())

```

Να μετατραπεί σε ισοδύναμο πρόγραμμα C++ με χρήση `vector<int>`.

Απάντηση:

```

#include <iostream>
#include <vector>
#include <string>
using namespace std;

class Student {
private:
    string name;
    vector<int> grades;

public:
    Student(string name, vector<int> grades) {
        this->name = name;
        this->grades = grades;
    }
};

```

```

    }

    double average() {
        int sum = 0;
        for (int grade : grades) {
            sum += grade;
        }
        return (double)sum / grades.size();
    }
};

int main() {
    Student student("Maria", {8, 9, 10});
    cout << student.average() << endl;
    return 0;
}

```

Ερώτημα 14

Δίνεται ο παρακάτω κώδικας Python:

```

class Person:
    def __init__(self, name):
        self.name = name

    def show(self):
        print("Name:", self.name)

class Student(Person):
    def __init__(self, name, department):
        super().__init__(name)
        self.department = department

    def show(self):
        print("Name:", self.name)
        print("Department:", self.department)

student = Student("Nikos", "Computer Science")
student.show()

```

Να μετατραπεί σε ισοδύναμο πρόγραμμα C++.

Απάντηση:

```

#include <iostream>
#include <string>
using namespace std;

class Person {
protected:
    string name;

public:
    Person(string name) {
        this->name = name;
    }

    void show() {
        cout << "Name: " << name << endl;
    }
}

```

```

    }
};

class Student : public Person {
private:
    string department;

public:
    Student(string name, string department) : Person(name) {
        this->department = department;
    }

    void show() {
        cout << "Name: " << name << endl;
        cout << "Department: " << department << endl;
    }
};

int main() {
    Student student("Nikos", "Computer Science");
    student.show();
    return 0;
}

```

Ερώτημα 15

Δίνεται ο παρακάτω κώδικας Python:

```

class Engine:
    def __init__(self, horsepower):
        self.horsepower = horsepower

    def get_specs(self):
        return f"{self.horsepower} hp"

class Car:
    def __init__(self, brand, engine):
        self.brand = brand
        self.engine = engine

    def show(self):
        print(self.brand, self.engine.get_specs())

engine = Engine(150)
car = Car("Ford", engine)
car.show()

```

Να μετατραπεί σε ισοδύναμο πρόγραμμα C++ όπου η κλάση Car περιέχει ένα αντικείμενο της κλάσης Engine.

Απάντηση:

```

#include <iostream>
#include <string>
using namespace std;

class Engine {
private:
    int horsepower;

```

```

public:
    Engine(int horsepower) {
        this->horsepower = horsepower;
    }

    string get_specs() {
        return to_string(horsepower) + " hp";
    }
};

class Car {
private:
    string brand;
    Engine engine;

public:
    Car(string brand, Engine engine) : engine(engine) {
        this->brand = brand;
    }

    void show() {
        cout << brand << " " << engine.get_specs() << endl;
    }
};

int main() {
    Engine engine(150);
    Car car("Ford", engine);
    car.show();
    return 0;
}

```

Κεφάλαιο 14 (Χειρισμός εξαιρέσεων και χειρισμός συμβάντων)

Ερώτημα 1

Να γράψετε πρόγραμμα σε Python που ζητά από τον χρήστη δύο αριθμούς και εμφανίζει το πηλίκο τους. Αν ο δεύτερος αριθμός είναι 0, να εμφανίζεται κατάλληλο μήνυμα λάθους χωρίς να τερματίζεται το πρόγραμμα. Παρατήρηση: να γίνει χρήση της εξαίρεσης `ZeroDivisionError`.

Απάντηση:

```
try:
    a = float(input("Δώσε τον πρώτο αριθμό: "))
    b = float(input("Δώσε τον δεύτερο αριθμό: "))
    quotient = a / b
    print("Το πηλίκο είναι:", quotient)

except ZeroDivisionError:
    print("Σφάλμα: Δεν επιτρέπεται η διαίρεση με το μηδέν.")
```

Ερώτημα 2

Να γράψετε πρόγραμμα σε Python που ζητά από τον χρήστη την ηλικία του. Το πρόγραμμα πρέπει να μετατρέπει την είσοδο σε ακέραιο αριθμό. Αν ο χρήστης πληκτρολογήσει κάτι που δεν είναι αριθμός, να εμφανίζεται μήνυμα λάθους. Παρατήρηση: να γίνει χρήση της εξαίρεσης `ValueError`.

Απάντηση:

```
try:
    age = int(input("Δώσε την ηλικία σου: "))
    print("Η ηλικία σου είναι:", age)
except ValueError:
    print("Σφάλμα: πρέπει να δώσεις ακέραιο αριθμό.")
```

Ερώτημα 3

Δίνεται η λίστα:

```
numbers = [10, 20, 30, 40]
```

Να γράψετε πρόγραμμα σε Python που ζητά από τον χρήστη έναν δείκτη θέσης και εμφανίζει το αντίστοιχο στοιχείο της λίστας. Αν ο δείκτης είναι εκτός ορίων, να εμφανίζεται κατάλληλο μήνυμα. Παρατήρηση: να γίνει χρήση της εξαίρεσης `IndexError`.

Απάντηση:

```
numbers = [10, 20, 30, 40]

try:
    index = int(input("Δώσε δείκτη θέσης: "))
    print(numbers[index])
except IndexError:
    print("Σφάλμα: ο δείκτης είναι εκτός ορίων.")
```

Ερώτημα 4

Να γράψετε πρόγραμμα σε Python που ζητά από τον χρήστη το όνομα ενός αρχείου και προσπαθεί να το ανοίξει για ανάγνωση. Αν το αρχείο δεν υπάρχει, να εμφανίζεται μήνυμα ότι το αρχείο δεν βρέθηκε. Παρατήρηση: να γίνει χρήση της εξαίρεσης `FileNotFoundError`.

Απάντηση:

```
filename = input("Δώσε όνομα αρχείου: ")

try:
```

```
with open(filename, "r") as file:
    print(file.read())
except FileNotFoundError:
    print("Σφάλμα: το αρχείο δεν βρέθηκε.")
```

Ερώτημα 5

Στην Python γράψτε μια συνάρτηση `calculate(a, b, operator)` που λαμβάνει δύο αριθμούς `a` και `b` και έναν τελεστή ως λεκτικό ('+' ή '/'). Να διασφαλίζει ότι οι είσοδοι `a` και `b` είναι αριθμοί, αν όχι να προκαλείται ένα `ValueError` με κατάλληλο μήνυμα. Να εκτελεί την αριθμητική πράξη με βάση τον τελεστή που περνά, αν ο τελεστής δεν είναι κάποιος από τους '+' ή '/' τότε να προκαλείται ένα `ValueError` με κατάλληλο μήνυμα. Να χειρίζεται την περίπτωση διαίρεσης με μηδέν συλλαμβάνοντας την εξαίρεση `ZeroDivisionError`. Ειδικότερα να εμφανίζονται οι έξοδοι όπως φαίνεται στη συνέχεια:

```
print(calculate(10, 5, '+')) # 15.0
print(calculate(10, 5, '/')) # 2.0
print(calculate(10, 0, '/')) # Σφάλμα: Διαίρεση με μηδέν.
print(calculate(10, 'a', '+')) # Σφάλμα: Λάθος είσοδος. Εισάγετε μόνο αριθμούς.
print(calculate(10, 5, '*')) # Σφάλμα: Λάθος τελεστής. Χρησιμοποιήστε μόνο '+' ή '/'.
```

Απάντηση:

```
def calculate(a, b, operator):
    try:
        # Μετατροπή εισόδων σε float και έλεγχος για ValueError
        a = float(a)
        b = float(b)
    except ValueError:
        return "Σφάλμα: Λάθος είσοδος. Εισάγετε μόνο αριθμούς."

    try:
        if operator == '+':
            return a + b
        elif operator == '/':
            return a / b
        else:
            raise ValueError("Λάθος τελεστής.")
    except ZeroDivisionError:
        return "Σφάλμα: Διαίρεση με μηδέν."
    except ValueError as e:
        return f"Σφάλμα: {e} Χρησιμοποιήστε μόνο '+' ή '/'."

# Έλεγχος της συνάρτησης calculate
print(calculate(10, 5, '+'))
print(calculate(10, 5, '/'))
print(calculate(10, 0, '/'))
print(calculate(10, 'a', '+'))
print(calculate(10, 5, '*'))
```

Ερώτημα 6

Σχεδιάστε το τι θα εμφανίσει ο ακόλουθος κώδικας και περιγράψτε το ποια θα είναι η λειτουργικότητά του.

```
import tkinter as tk

def increment():
    x = int(count_label["text"])
    x += 1
    count_label["text"] = str(x)
```

```

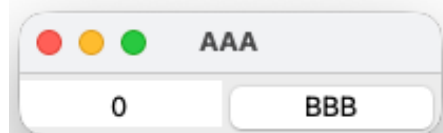
root = tk.Tk()
root.geometry("200x30")
root.title("AAA")
count_label = tk.Label(root, text="0", width=10, background="white")
count_label.pack(side=tk.LEFT)
increment_button = tk.Button(root, text="BBB", width=10)
increment_button.pack(side=tk.RIGHT)
increment_button.config(command=increment)

root.mainloop()

```

Απάντηση:

Θα εμφανίσει ένα παράθυρο με μια ετικέτα με αρχική τιμή 0 και ένα πλήκτρο, όπως στο ακόλουθο σχήμα.



Κάθε φορά που ο χρήστης θα πατά το πλήκτρο η τιμή στην ετικέτα θα αυξάνεται κατά 1.

Ερώτημα 7

Δίνεται το παρακάτω πρόγραμμα που δημιουργεί ένα γραφικό περιβάλλον με χρήση του tkinter:

```

import tkinter as tk

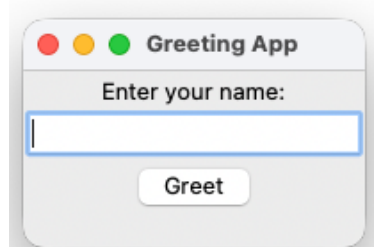
root = tk.Tk()
root.title("Greeting App")
label = tk.Label(root, text="Enter your name:")
label.pack()
entry = tk.Entry(root)
entry.pack()
def greet():
    name = entry.get()
    greeting.config(text=f"Hello, {name}!")
button = tk.Button(root, text="Greet", command=greet)
button.pack()
greeting = tk.Label(root, text="")
greeting.pack()
root.mainloop()

```

Να σχεδιάσετε το GUI που εμφανίζεται και να περιγράψετε τη συμπεριφορά του.

Απάντηση:

Θα εμφανίσει ένα παράθυρο με μια ετικέτα με κείμενο "Enter your name:", ένα πεδίο εισαγωγής κειμένου, ένα πλήκτρο με κείμενο "Greet" και μια ετικέτα χωρίς περιεχόμενο, όπως στο ακόλουθο σχήμα:



Όταν ο χρήστης πατά το πλήκτρο εμφανίζεται ως κείμενο στην ετικέτα το "Hello" ακολουθούμενο από το κείμενο του πεδίου εισαγωγής κειμένου και ένα θαυμαστικό.

Ερώτημα 8

Δίνεται το παρακάτω πρόγραμμα που δημιουργεί ένα γραφικό περιβάλλον με χρήση του tkinter:

```

import tkinter as tk

```

```

window = tk.Tk()
window.title("Add-Subtract Calculator")
frame = tk.Frame(window, borderwidth=2, relief='groove')
frame.pack(padx=10, pady=10)
entry1 = tk.Entry(frame, width=10)
entry1.grid(row=0, column=0, padx=5)
entry2 = tk.Entry(frame, width=10)
entry2.grid(row=0, column=1, padx=5)
result_label = tk.Label(window, text="Result: -")
result_label.pack(pady=5)

def add():
    total = int(entry1.get()) + int(entry2.get())
    result_label.config(text=f"Result: {total}")

def subtract():
    diff = int(entry1.get()) - int(entry2.get())
    result_label.config(text=f"Result: {diff}")

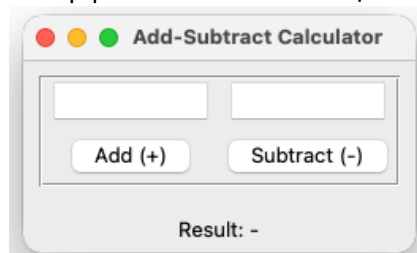
button_add = tk.Button(frame, text="Add (+)", command=add)
button_add.grid(row=1, column=0, pady=5)
button_subtract = tk.Button(frame, text="Subtract (-)", command=subtract)
button_subtract.grid(row=1, column=1, pady=5)
window.mainloop()

```

Να σχεδιάσετε το GUI που εμφανίζεται και να περιγράψετε τη συμπεριφορά του.

Απάντηση:

Θα εμφανίσει ένα GUI όπως το ακόλουθο:



Ανάλογα με το πλήκτρο που θα πατιέται, υπολογίζεται το άθροισμα ή η αφαίρεση των 2 τιμών που ο χρήστης θα έχει εισάγει στα πεδία εισαγωγής κειμένου και το αποτέλεσμα εμφανίζεται στην ετικέτα στο κάτω μέρος της οθόνης.

Ερώτημα 9

Δίνεται το παρακάτω πρόγραμμα που δημιουργεί ένα γραφικό περιβάλλον με χρήση του tkinter:

```

import tkinter as tk

window = tk.Tk()
window.title("Notes App")
frame = tk.Frame(window, borderwidth=2, relief="groove")
frame.pack(padx=10, pady=10)
entry = tk.Entry(frame, width=20)
entry.grid(row=0, column=0, padx=5, pady=5)
listbox = tk.Listbox(window, width=30, height=5)
listbox.pack(pady=10)
def add_note():
    note = entry.get()
    if note:
        listbox.insert(tk.END, note)
        entry.delete(0, tk.END)
button_add = tk.Button(frame, text="Add Note", command=add_note)

```

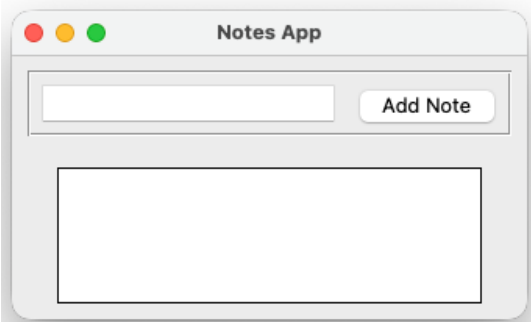


```
button_add.grid(row=0, column=1, padx=5, pady=5)
window.mainloop()
```

Να σχεδιάσετε το GUI που εμφανίζεται και να περιγράψετε τη συμπεριφορά του.

Απάντηση:

Θα εμφανίσει ένα GUI όπως το ακόλουθο:



Ο χρήστης εισάγει ένα κείμενο στο πεδίο εισαγωγής κειμένου πάνω αριστερά και πατώντας το πλήκτρο “Add note” το κείμενο εισάγεται στη λίστα στο κάτω μέρος της οθόνης.